

Session 13 : Les moteurs physiques commerciaux

Objectifs de la session

- Comprendre **pourquoi** on utilise un moteur physique existant en production plutôt que le sien.
- Découvrir les moteurs majeurs (**PhysX, Havok, Jolt, Box2D, Rapier**) et leurs cas d'usage.
- Découvrir les concepts qu'ils partagent : **CCD, sleeping, joints, query** (raycast).
- Comprendre le **déterminisme** et pourquoi il est difficile à obtenir.
- Introduire l'architecture **ECS** (Entity-Component-System) qui structure les moteurs modernes.
- **Préparer la pratique** (Session 14) : installer **Rapier** avec Three.js et activer **Jolt** dans Godot.

💡 Le moment charnière du cours

Depuis la Session 1, vous construisez **votre propre moteur** : intégrateurs, collisions, impulsions, ressorts, particules. Vous savez maintenant **pourquoi c'est difficile**. Cette session répond à la question : **“Et en production, on refait tout ça ?”** — Non. On utilise un moteur éprouvé. Mais maintenant, vous **comprenez ce qu'il fait sous le capot**, et c'est ce qui distingue un programmeur de jeux qui **utilise** un moteur d'un programmeur qui le **comprend**.

Partie 1 : Pourquoi utiliser un moteur physique ?

Le problème de la complexité

Notre moteur “maison” couvre des cas simples : quelques dizaines d'objets, formes convexes, impulsions basiques. Un jeu réel exige :

- **Des centaines d'objets** en interaction simultanée (empilements, destructions).
- **Des formes complexes** : convexes, concaves (meshes), composées.
- **De la stabilité** : un empilement de cubes ne doit pas trembler ni exploser.
- **Des joints** : charnières, ressorts, moteurs (ragdolls, véhicules).
- **Des queries** : raycasts pour les tirs, shapecasts pour les personnages.
- **De la performance** : tout ça en moins de 2 ms par frame.

Avantages des moteurs existants

- **Complexité** : gestion des contacts multiples, stabilité des empilements, solveurs itératifs optimisés.
- **Temps** : des décennies de R&D cumulées (PhysX a plus de 20 ans).
- **Performance** : SIMD, multithreading, broad phase optimisée — des milliers d'objets à 60 FPS.
- **Outils** : debug rendering, serialization, détermination des scènes.
- **Communauté** : bugs connus, solutions documentées, intégrations prêtes.

💬 Mais pourquoi on a fait notre moteur alors ?

Parce qu'un moteur physique est une **boîte noire dangereuse** si on ne comprend pas ce qu'elle contient. Sans les Sessions 1–12, un moteur commercial est magique : **“pourquoi mon personnage traverse le sol ?”, “pourquoi mon empilement tremble ?”, “pourquoi mes objets flottent ?”**. Avec elles, vous savez que c'est une question de **timestep**, de **solver iterations**, de **sleeping threshold** — et vous savez **où regarder**.

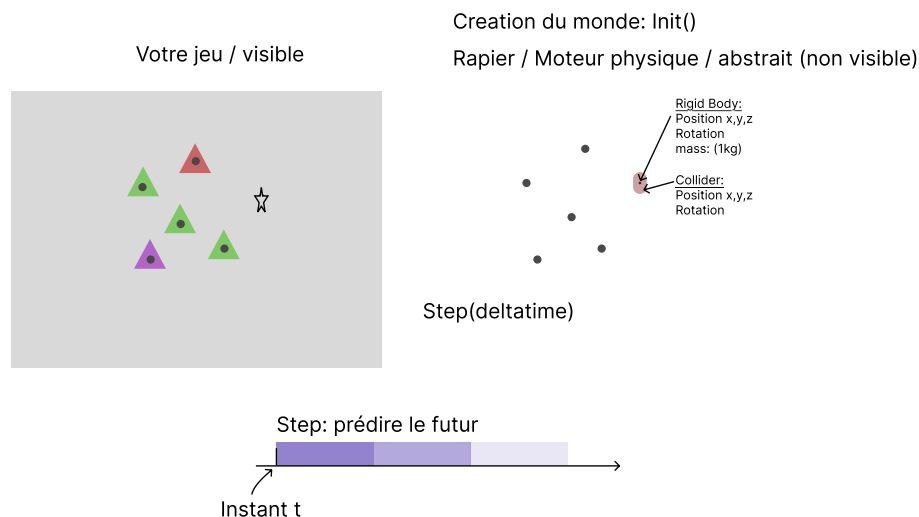


Figure 1: Un moteur physique commercial : on fournit les colliders et les forces, il retourne les positions et rotations. Toute la machinerie interne — broad phase, narrow phase, solveur de contraintes — est invisible mais fonctionne sur les principes vus en cours.

Partie 2 : Concepts partagés par tous les moteurs

Rigid Body vs Soft Body

Rigid Body vs Soft Body

- **Rigid Body** : objet **indéformable** (boîte, sphère, capsule). La distance entre deux points internes ne change jamais. C'est ce que tous les moteurs font nativement — et tout ce dont un jeu a besoin dans 95 % des cas.
- **Soft Body** : objet **déformable** (tissu, gelée, ballon). La forme change en fonction des forces appliquées. Beaucoup plus coûteux — souvent simulé par un système masse-ressort comme notre tissu de la Session 8, ou par des techniques spécialisées (FEM, position-based dynamics).

💡 La stratégie classique

Les jeux utilisent des **rigid bodies** presque partout, et simulent la déformation **visuellement** : un ragdoll est un ensemble de rigid bodies reliés par des joints ; une destruction est un échange d'un mesh statique contre plusieurs rigid bodies. La déformation "vraie" (soft body) est réservée aux cas où elle est le gameplay (World of Goo, JellyCar).

Continuous Collision Detection (CCD)

Le problème du tunneling

Un objet rapide peut **traverser** un mur mince entre deux frames. Si une balle va à 100 m/s avec un dt de 1/60 s, elle se déplace de **1,67 m par pas**. Un mur de 0,2 m d'épaisseur ? La balle est devant le mur à t , derrière à $t + dt$ — aucune des deux positions n'est **dans** le mur. Le test discret (Session 6) ne voit jamais la collision.

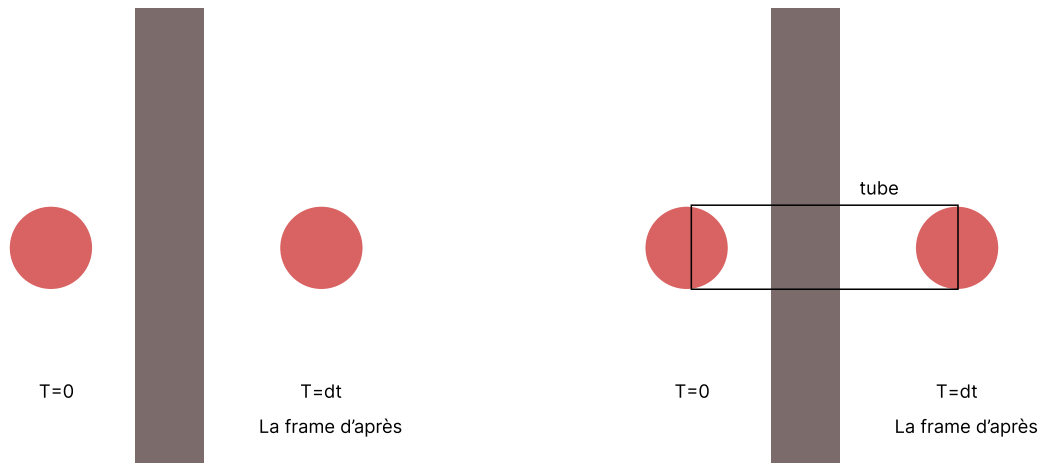


Figure 2: Tunneling : la balle rapide passe de l'autre côté du mur entre deux frames. La détection discrète (positions à t et $t + dt$) rate la collision. Le CCD teste la **trajectoire complète** entre t et $t + dt$.

La solution : CCD

Le **Continuous Collision Detection** teste la collision le long de la **trajectoire** entre t et $t + dt$, pas seulement aux positions discrètes :

- **Swept test** : on “balaie” la forme le long de son déplacement et on teste le volume couvert.
- **Raycast** : pour une sphère, on raycaste le centre le long de $\vec{v} \cdot dt$ avec un rayon élargi.
- **Conservative advancement** : on avance par petits pas adaptatifs jusqu’au premier contact.

Coût : le CCD est **plus cher** que la détection discrète. C’est pourquoi les moteurs l’activent **par objet** — uniquement pour les projectiles rapides, pas pour tous les objets.

CCD dans les moteurs.

- **PhysX / Unity** : `rigidbody.enableCcd = true` — pour les projectiles.
- **Rapier** : `RigidBodyDesc.dynamic().setCcdEnabled(true)`.
- **Godot** : propriété `continuous_cd` sur les `RigidBody`.
- **Notre moteur** : le sub-stepping de la Session 12 est une **rustine approximative** de CCD — on réduit dt pour que le déplacement par pas soit petit devant les obstacles.

Sleeping

Sleeping (mise en sommeil)

Si un objet est **immobile** (vitesse et vitesse angulaire sous un seuil pendant un certain temps), on arrête de le simuler. Il “dort” :

- Il ne consomme plus de temps de calcul.
- Il ne réagit plus aux forces tant qu’il dort.
- Il se **réveille** si un autre objet le touche, ou si on lui applique une force programmatique.

Pourquoi c’est essentiel : dans un niveau, 90 % des objets physiques sont au sol et immobiles. Sans sleeping, on simule inutilement des milliers de corps stables — le broad phase tourne, le solveur itère, et tout ça pour rien.

! Le piège du sleeping

Un objet endormi ne détecte plus les collisions **provoquées par lui**. Si un personnage marche sur un pont d'objets endormis, il faut que le premier contact **réveille** la chaîne. Les moteurs gèrent ça automatiquement — mais un objet au-dessus du sol qui “flotte” endormi est un bug classique quand on téléporte des objets sans les réveiller.

Les types de corps

Les trois types de rigid bodies

Tous les moteurs distinguent :

- **Dynamic** : subit les forces et les collisions. Masse finie. C'est l'objet physique standard.
- **Fixed (Static)** : ne bouge **jamais**. Masse infinie. Le sol, les murs, le décor.
- **Kinematic** : contrôlé **par le code** (position ou vitesse), ignore les forces, mais pousse les dynamic bodies. Plateformes mobiles, ascenseurs.

La distinction existe parce que simuler un mur comme un dynamic body de masse énorme coûte cher et cause de l'instabilité numérique. Un fixed body est **gratuit** et **parfaitement stable**.

Type	Subit les forces ?	Pousse les dyn. ?	Usage
Dynamic	Oui	Oui	Objets du jeu, débris, projectiles
Fixed	Non	Oui (mur)	Sol, murs, niveau
Kinematic	Non (code !)	Oui	Plateformes, ascenseurs, portes

Table 1: Les trois types de corps. Le kinematic est le plus subtil : le code contrôle sa trajectoire, mais les dynamic bodies réagissent quand il les pousse.

Les queries (raycast, shapecast)

Scene queries

Au-delà de la simulation, les moteurs offrent des **questions directes** sur l'état du monde :

- **Raycast** : “qu'est-ce que cette droite touche, et à quelle distance ?” — essentiel pour les tirs (hitscan), la ligne de vue, les IA.
- **Shapecast** : “si je déplace cette forme de A à B, que touche-t-elle en premier ?” — utilisé par les character controllers pour se déplacer sans traverser les murs.
- **Overlap** : “quels corps chevauchent ce volume ?” — zones de dégâts, pickups, déclencheurs.

Les queries **ne modifient pas** la simulation — elles l'interrogent.

? Le raycast, outil universel

Le personnage ne “marche” presque jamais avec un rigid body — il fait un **shapecast vers le bas** pour trouver le sol, puis le code déplace le personnage et résout les collisions manuellement. C'est pour ça que les moteurs ont des **CharacterBody** (Godot, Unity) : des corps non simulés qui utilisent les queries pour se déplacer de façon contrôlée. On les verra en Session 18.

Partie 3 : Le Paysage des Moteurs

Les moteurs natifs (C/C++)

PhysX — le standard historique

Développé par NVIDIA (originellement NovodeX, 2001). Open-source depuis 2016. C’est le moteur **par défaut** de Unity et Unreal Engine. On le trouve dans la plupart des AAA. Points forts : maturité, outils, GPU acceleration (pour la simulation massive). C’est un **standard de facto**.

Havok — le moteur premium

Moteur commercial fermé (Irlande, 2000). Utilisé par Half-Life 2, Zelda TotK, l’ensemble des grosses productions Sony/Microsoft. Réputé pour sa **stabilité** sur les empilements massifs et sa destruction avancée. Coûteux — réservé aux studios avec un budget.

Jolt Physics — le challenger moderne

Open-source (par Jorrit Rouwe, 2021), écrit en C++ moderne. Rapide, multithreadé, déterministe. Utilisé pour **Horizon Forbidden West** (démonstration de puissance), et il est devenu le moteur **par défaut** de Godot 4.4+. Le succès récent le plus notable du domaine.

Box2D — le roi du 2D

D’Erin Catto (2006), open-source. La référence 2D : Angry Birds, Terraria, et des centaines de jeux 2D. Le solveur de contraintes de Box2D (Sequential Impulse) est **le** solveur que tous les autres ont copié. Le moteur “maison 2D” de ce cours est architecturé comme un mini-Box2D.

Les moteurs web (JavaScript / WASM)

Rapier — le standard web moderne

Écrit en **Rust**, compilé en **WebAssembly**. Rapide, déterministe, API moderne. Le moteur **recommandé** pour Three.js. Développé par Dimforge. Nous allons l’utiliser aujourd’hui. Points forts : performance proche du natif, WASM (le code binaire tourne à pleine vitesse dans le navigateur), API propre.

Cannon.js / Ammo.js — la génération précédente

- **Cannon.js** : moteur en pur JavaScript (2011), simple mais lent, peu maintenu. Cannon-es est un fork maintenu.
- **Ammo.js** : portage Emscripten de Bullet Physics vers WASM. Performant mais API datée, gros bundle (2 Mo), moins pratique.

Ces moteurs ont précédé Rapier — aujourd’hui, pour un nouveau projet Three.js, **Rapier est le choix par défaut**.

Moteur	Langage	Licence	Usage notable
PhysX	C++	Open source	Unity/Unreal par défaut, AAA
Havok	C++	Commerciale	Half-Life 2, Zelda TotK
Jolt Physics	C++	Open source	Horizon FW, Godot 4.4+
Box2D	C++	Open source	Angry Birds, Terraria (2D)
Bullet	C++	Open source	Blender, GTA IV
Rapier	Rust→WASM	Open source	Web (Three.js), Bevy
Cannon-es	JavaScript	Open source	Three.js (simple, legacy)
Ammo.js	C++→WASM	Open source	Three.js (Bullet port)
Godot Physics	C++	Open source	Godot (Godot Physics/Jolt)

Table 2: Paysage des moteurs physiques. Notez Godot en bas : le moteur de jeu **intègre** son propre moteur physique (ou Jolt) — pas besoin d’en choisir un.

💡 Le retour de Jolt — une leçon d'humilité

Jolt a été écrit **par une seule personne** (Jorrit Rouwe) en quelques années, et il rivalise avec PhysX et Havok, développés par des équipes entières depuis 20 ans. Il a été adopté par Godot comme moteur par défaut. Preuve que le domaine n’est pas figé — et qu’une **compréhension profonde** des principes (les vôtres maintenant) permet de construire des outils de classe mondiale.

Partie 4 : Déterminisme

Déterminisme

Un moteur est **déterministe** si les mêmes inputs produisent **toujours exactement les mêmes outputs** — même sur des machines différentes, même après des millions de pas.

💬 Pourquoi c'est important

- **Multijoueur lockstep** : chaque client simule la même physique et n’échange que les inputs. Si la physique diverge d’un bit, les joueurs voient des mondes **différents** — le jeu casse. StarCraft II, Age of Empires IV fonctionnent comme ça.
- **Replays** : enregistrer seulement les inputs et rejouer la simulation requiert un moteur déterministe.
- **Tests de régression** : un bug de physique se reproduit exactement — on peut le déboguer.
- **Rollback netcode** : rembobiner la physique et rejouer (fighting games) exige le déterminisme.

Pourquoi c'est difficile

Le déterminisme se casse par :

- **Ordre des opérations** : si les paires de collision sont traitées dans un ordre différent (hash map, multithreading), les résultats divergent.
- **Précision flottante** : le format IEEE 754 est déterministe pour $+$, $-$, $\times$, $\div$ — mais pas pour sqrt , sin , cos (implémentations différentes selon le CPU/libm).
- **Multithreading** : deux threads qui traitent des corps en interaction dans un ordre non fixé $\rightarrow$ divergence.

Solutions : ordre de traitement **fixe et trié** (par handle, pas par adresse), maths à précision **fixée** (fixed-point ou entiers), fonctions trigonométriques **réimplémentées** (deterministic sin/cos), thread scheduling **constraint**.

⚠ En pratique

Rapier est déterministe **localement** (même machine, même binaire) mais pas **cross-platform** par défaut. Pour du lockstep multijoueur en production, on utilise des moteurs spécifiquement conçus pour ça (ou on fixe la précision manuellement). Godot **n'est pas** déterministe — les replays Godot enregistrent les états, pas les inputs.

Partie 5 : Architecture ECS

Entity-Component-System

L'architecture dominante des moteurs modernes (Unity DOTS, Bevy, Flecs, EnTT) :

- **Entity** : un simple **ID** (ex: `entity_42`). Pas de données, pas de logique. Juste une étiquette.
- **Component** : des **données pures** (ex: `Position`, `RigidBody`, `Mesh`). Un composant ne contient **aucune logique** — juste des champs.
- **System** : de la **logique pure** qui opère sur toutes les entités possédant certains composants (ex: `PhysicsSystem` traite tout ce qui a `RigidBody` + `Position`).

L'idée en code.

```
// Entity: juste un ID
const entity = world.createEntity();      // entity = 42

// Components: des données pures
world.addComponent(entity, Position, { x: 0, y: 10, z: 0 });
world.addComponent(entity, Velocity, { x: 0, y: 0, z: 0 });
world.addComponent(entity, RigidBody, { mass: 1.0 });

// System: de la logique qui traite toutes les entités
// possédant Position + Velocity
world.addSystem((dt) => {
    for (const e of world.query(Position, Velocity)) {
        e.position.x += e.velocity.x * dt;    // ...
    }
});
```

💡 Pourquoi ECS plutôt que l'héritage ?

L'approche orientée objet classique (`class Projectile extends PhysiqueObject extends GameObject`) crée des **hiérarchies rigides** : un objet “posable ET physique ET dégâts” force des contorsions d'héritage.

ECS compose : un baril = Position + RigidBody + Mesh + Pickup. Une flèche = Position + RigidBody + Mesh + Projectile. On **ajoute et enlève des composants à la volée** — un ennemi mort devient un ragdoll : on retire AI, on ajoute RigidBody à ses membres.

Bonus majeur : les **systems** itèrent sur des tableaux de données contiguës (**cache-friendly**) — c'est ce qui rend les moteurs ECS si rapides.

💡 Le lien avec la physique

Rapier, en interne, ressemble à un ECS : chaque RigidBody est une entité avec des composants (position, velocity, collider handles). Quand on écrit `world.step()`, un `PhysicsSystem` itère sur tous les corps. Vous avez **déjà** écrit ce code — votre boucle `updatePhysics(dtFrame)` de la Session 12 est un `PhysicsSystem` !

Partie 6 : Choisir son moteur — le guide pratique

Contexte	Choix recommandé	Pourquoi
Jeu web Three.js	Rapier	WASM rapide, API moderne, le standard
Jeu web 2D simple	Rapier 2D ou Planck.js	Rapide et simple
Jeu Godot	Intégré (Jolt)	Zéro intégration, outils inclus
Jeu Unreal	Chaos (intégré)	Intégré + destruction
Jeu Unity	PhysX (défaut)	Intégré, mature
Simulation massive	PhysX GPU / Jolt	Multithread, GPU
Multijoueur lockstep	Moteur déterministe dédié	Floats cross-platform

Table 3: Guide de choix rapide. Dans tous les cas : les concepts sont les mêmes, seule l'API change.

Synthèse

💡 Ce qu'il faut retenir

- Un **moteur physique commercial** économise des années de R&D — mais sans comprendre les principes (intégration, collisions, solveur), on ne peut pas le déboguer ni le pousser dans ses retranchements.
- Tous les moteurs partagent les mêmes concepts : **rigid bodies** (dynamic/fixed/kinematic), **colliders**, **CCD** anti-tunneling, **sleeping**, **queries** (raycast).
- Le **pattern d'intégration** est universel : créer le monde, créer mesh + body par objet, `step()`, synchroniser. La physique est la source de vérité.

- **Le déterminisme** (mêmes inputs → mêmes outputs) est essentiel pour le multijoueur lockstep et les replays, mais difficile (ordre des opérations, flottants, threads).
- **L'ECS** (Entity-Component-System) est l'architecture des moteurs modernes : composition par données, logique dans les systems, cache-friendly.
- **Rapier** est le standard pour Three.js ; **Godot** intègre sa propre physique (Jolt) — deux philosophies qu'on mettra en pratique en Session 14.

💡 La suite — la pratique

Cette session a posé le **paysage** et les **concepts**. La **Session 14** est la partie **pratique** : on installe et configure **Rapier** avec Three.js et **Jolt** avec Godot, puis on construit **pas à pas** des rigid bodies et des colliders dans chaque moteur — setup, boucle de simulation, synchronisation, et le tableau comparatif des API. Tout ce qu'on a vu en théorie ici devient du code là-bas.